Source: Google Cloud Documentation, *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*
Section: MLOps Level 1: ML pipeline automation – Additional components

Original source: Google Cloud Architecture Center
Content reproduced from a larger document and provided for educational use.

Contributors:
Authors:
Jarek Kazmierczak | Solutions Architect
Khalid Salama | Staff Software Engineer, Machine Learning
Valentín Huerta | AI Engineer
Other contributor:
Sunil Kumar Jang Bahadur | Customer Engineer

Additional components

This section discusses the components that you need to add to the architecture to enable ML continuous training.

Data and model validation

When you deploy your ML pipeline to production, one or more of the triggers discussed in the ML pipeline triggers section automatically executes the pipeline. The pipeline expects new, live data to produce a new model version that is trained on the new data (as shown in Figure 3). Therefore, automated *data validation* and *model validation* steps are required in the production pipeline to ensure the following expected behavior:

- Data validation: This step is required before model training to decide whether you should retrain the model or stop the execution of the pipeline. This decision is automatically made if the following was identified by the pipeline.
    - Data schema skews: These skews are considered anomalies in the input data. Therefore, input data that doesn't comply with the expected schema is received by the downstream pipeline steps, including the data processing and model training steps. In this case, you should stop the pipeline so the data science team can investigate. The team might release a fix or an update to the pipeline to handle these changes in the schema. Schema skews include receiving unexpected features, not

receiving all the expected features, or receiving features with unexpected values.
    - Data values skews: These skews are significant changes in the statistical properties of data, which means that data patterns are changing, and you need to trigger a retraining of the model to capture these changes.
- Model validation: This step occurs after you successfully train the model given the new data. You evaluate and validate the model before it's promoted to production. This *offline model validation* step consists of the following.
    - Producing evaluation metric values using the trained model on a test dataset to assess the model's predictive quality.
    - Comparing the evaluation metric values produced by your newly trained model to the current model, for example, production model, baseline model, or other business-requirement models. You make sure that the new model produces better performance than the current model before promoting it to production.
    - Making sure that the performance of the model is consistent on various segments of the data. For example, your newly trained customer churn model might produce an overall better predictive accuracy compared to the previous model, but the accuracy values per customer region might have large variance.
    - Making sure that you test your model for deployment, including infrastructure compatibility and consistency with the prediction service API.

In addition to offline model validation, a newly deployed model undergoes *online model validation*—in a canary deployment or an A/B testing setup—before it serves prediction for the online traffic.

Feature store

An optional additional component for level 1 ML pipeline automation is a feature store. A feature store is a centralized repository where you standardize the definition, storage, and access of features for training and serving. A feature store needs to provide an API for both high-throughput batch serving and low-latency real-time serving for the feature values, and to support both training and serving workloads.

The feature store helps data scientists do the following:

- Discover and reuse available feature sets for their entities, instead of re-creating the same or similar ones.
- Avoid having similar features that have different definitions by maintaining features and their related metadata.
- Serve up-to-date feature values from the feature store.
- Avoid training-serving skew by using the feature store as the data source for experimentation, continuous training, and online serving. This approach makes sure that the features used for training are the same ones used during serving:
    - For experimentation, data scientists can get an offline extract from the feature store to run their experiments.
    - For continuous training, the automated ML training pipeline can fetch a batch of the up-to-date feature values of the dataset that are used for the training task.
    - For online prediction, the prediction service can fetch in a batch of the feature values related to the requested entity, such as customer demographic features, product features, and current session aggregation features.
    - For online prediction and feature retrieval, the prediction service identifies the relevant features for an entity. For example, if the entity is a customer, relevant features might include age, purchase history, and browsing behavior. The service batches these feature values together and retrieves all the needed features for the entity at once, rather than individually. This retrieval method helps with efficiency, especially when you need to manage multiple entities.

Metadata management

Information about each execution of the ML pipeline is recorded in order to help with data and artifacts lineage, reproducibility, and comparisons. It also helps you debug errors and anomalies. Each time you execute the pipeline, the ML metadata store records the following metadata:

- The pipeline and component versions that were executed.
- The start and end date, time, and how long the pipeline took to complete each of the steps.
- The executor of the pipeline.
- The parameter arguments that were passed to the pipeline.

- The pointers to the artifacts produced by each step of the pipeline, such as the location of prepared data, validation anomalies, computed statistics, and extracted vocabulary from the categorical features. Tracking these intermediate outputs helps you resume the pipeline from the most recent step if the pipeline stopped due to a failed step, without having to re-execute the steps that have already completed.
- A pointer to the previous trained model if you need to roll back to a previous model version or if you need to produce evaluation metrics for a previous model version when the pipeline is given new test data during the model validation step.
- The model evaluation metrics produced during the model evaluation step for both the training and the testing sets. These metrics help you compare the performance of a newly trained model to the recorded performance of the previous model during the model validation step.

ML pipeline triggers

You can automate the ML production pipelines to retrain the models with new data, depending on your use case:

- On demand: Ad hoc manual execution of the pipeline.
- On a schedule: New, labeled data is systematically available for the ML system on a daily, weekly, or monthly basis. The retraining frequency also depends on how frequently the data patterns change, and how expensive it is to retrain your models.
- On availability of new training data: New data isn't systematically available for the ML system and instead is available on an ad hoc basis when new data is collected and made available in the source databases.
- On model performance degradation: The model is retrained when there is noticeable performance degradation.
- On significant changes in the data distributions (*concept drift*). It's hard to assess the complete performance of the online model, but you notice significant changes on the data distributions of the features that are used to perform the prediction. These changes suggest that your model has gone stale, and that needs to be retrained on fresh data.

Challenges

Assuming that new implementations of the pipeline aren't frequently deployed and you are managing only a few pipelines, you usually manually test the pipeline and its components. In addition, you manually deploy new pipeline implementations. You also submit the tested source code for the pipeline to the IT team to deploy to the target environment. This setup is suitable when you deploy new models based on new data, rather than based on new ML ideas.

However, you need to try new ML ideas and rapidly deploy new implementations of the ML components. If you manage many ML pipelines in production, you need a CI/CD setup to automate the build, test, and deployment of ML pipelines.